

Predicting Implied Volatility and Options Pricing Analysis

Introduction and Problem Statement

This project attempts to implement a Machine Learning model on the \$SPY Options Chains dataset to accurately predict the Implied Volatility (IV). The model uses the following features to predict the IV – Strike Price, Strike Distance, Date to Expiry, Option volume / Open Interest, Underlying Price and Greeks (Delta, Gamma, Vega and Theta). The model uses **Random Forest Regression** to best predict the Implied Volatility of S&P 500 Call Options from 2021. The dataset includes an c_{iv} column that is used as the target variable. In the **Black-Scholes equation** shown below, the Implied Volatility is assumed to be constant. Since the real market does not behave as such, the results of Black-Scholes pricing can differ from the actual market prices. To address this, the main goal of the project is to predict the Implied Volatility to be able to accurately price options using the same Black-Scholes model but by swapping out the constant IV with the predicted values.

$$C = N(d_1)S_t - N(d_2)Ke^{-rt}$$

$$\text{where } d_1 = \frac{\ln \frac{S_t}{K} + (r + \frac{\sigma^2}{2})t}{\sigma\sqrt{t}}$$

$$\text{and } d_2 = d_1 - \sigma\sqrt{t}$$

C = call option price

N = CDF of the normal distribution

S_t = spot price of an asset

K = strike price

r = risk-free interest rate

t = time to maturity

σ = volatility of the asset

Dataset - [\\$SPY Option Chains - Q1 2020 - Q4 2022](#)

The above dataset has the S&P500 Index Call and Put options data for Q1 2020 – Q4 2022.

Although the dataset has 3.5 million rows and 33 columns, the model will use the first 40000 rows to reduce computation and make the scope feasible. Furthermore, this model will focus solely on Call options and ignore Put options entirely to avoid conflicting data.

Methodology and Data Pre-processing steps:

Data wrangling

1. Remove punctuation from column names to make the workflow easier

2. Drop redundant columns –

figure a. `c_last`, `c_bid`, `c_ask` are prices used to compute the `c_iv`

figure b. `c_size`, `'quote_readtime'`, `'quote_date'`, `'quote_time_hours'`,
`'quote_unixtime'`, `'expire_unix'` - Redundant metadata

figure c. All put option features since this model will focus on call options

3. Convert Object type columns to float64

4. Delete rows with missing data using `isnull()`

5. Add derived features -

figure a. Moneyness ($\text{Underlying_last} / \text{Strike ratio}$)

figure b. Log moneyness

figure c. Time to expiry ($\text{tte} = \text{dte} * 365$)

6. Remove Outliers using IQR

7. Managing skewness – Visualizing the target variable IV on a histogram showed heavy right skewness in data. To address this, the model uses log transformation on IV

8. Normalize features using standard scaler (after splitting test and train sets)

Data Visualization

figure a. Histogram of target variable c_iv (Call Implied Volatility)

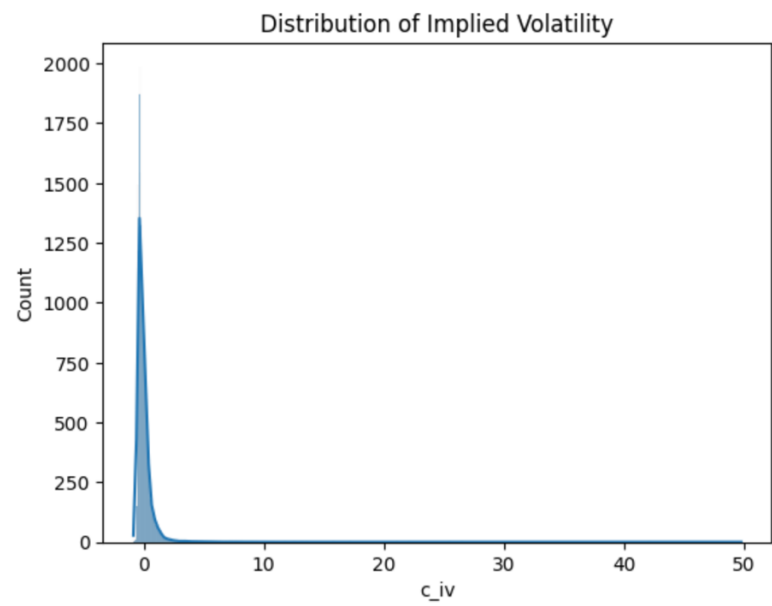


figure b. Scatter plot of IV vs DTE

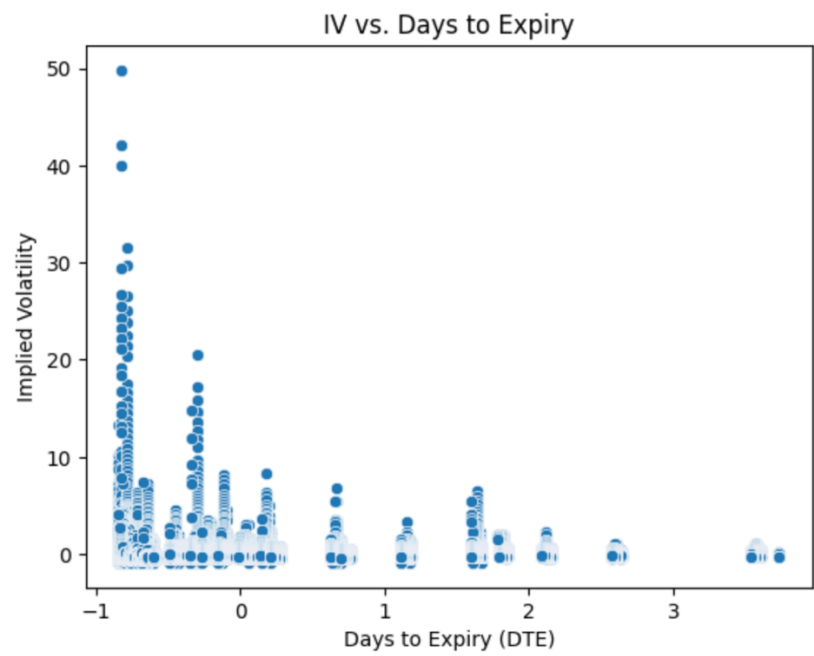


figure c. Correlation heatmap

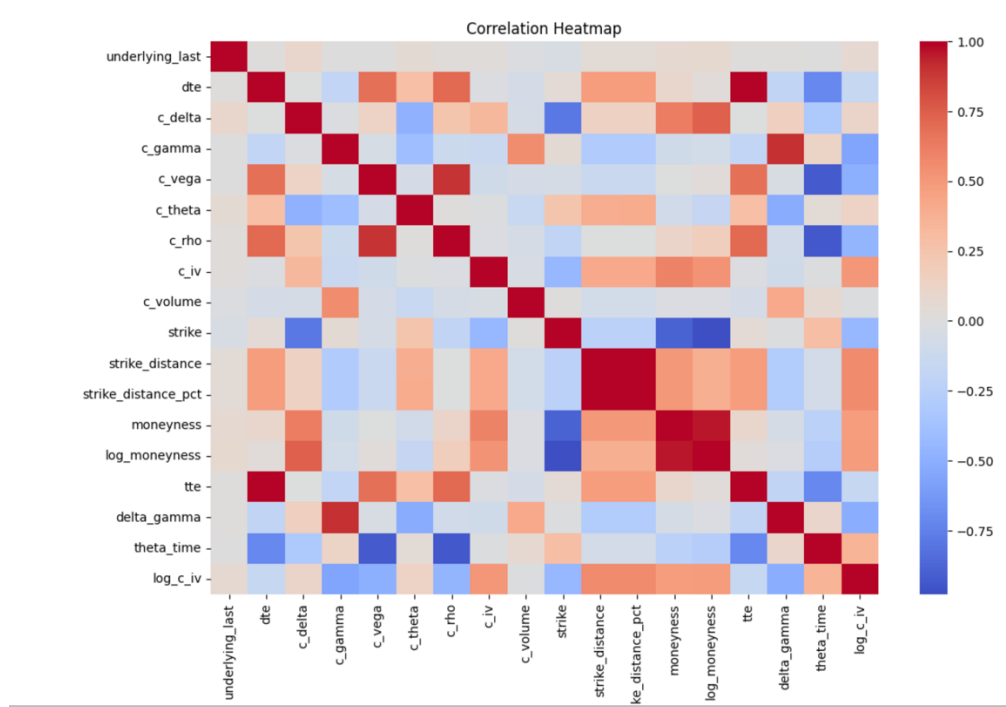


figure d. Histogram of **log-transformed IV** (reduced skew)

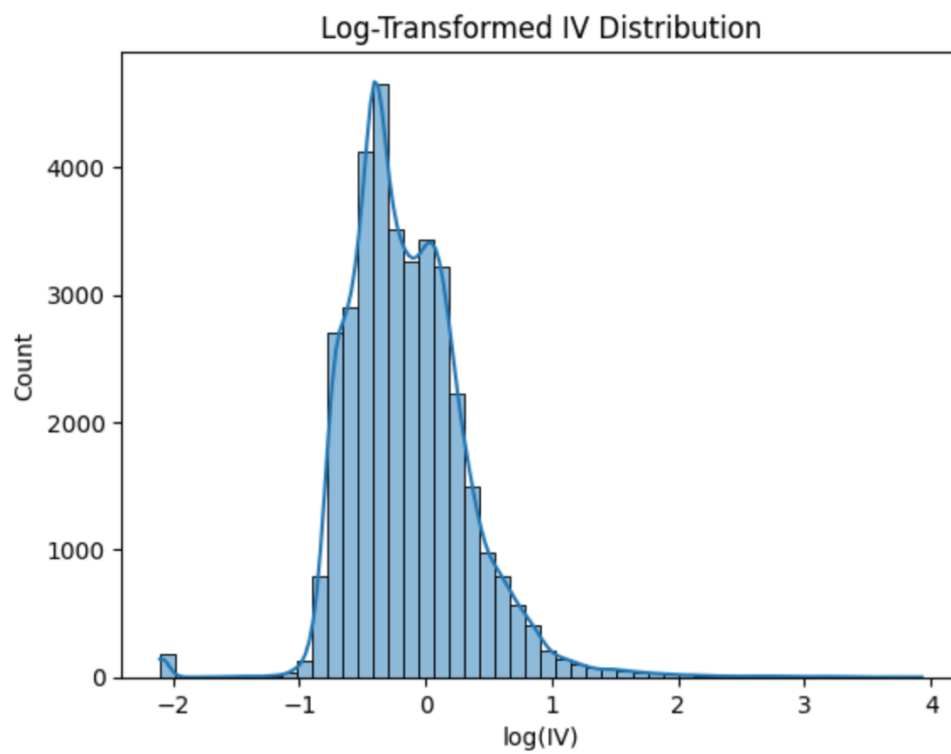


figure e. Distplot (Seaborn) of Actual vs Predicted values of log IV

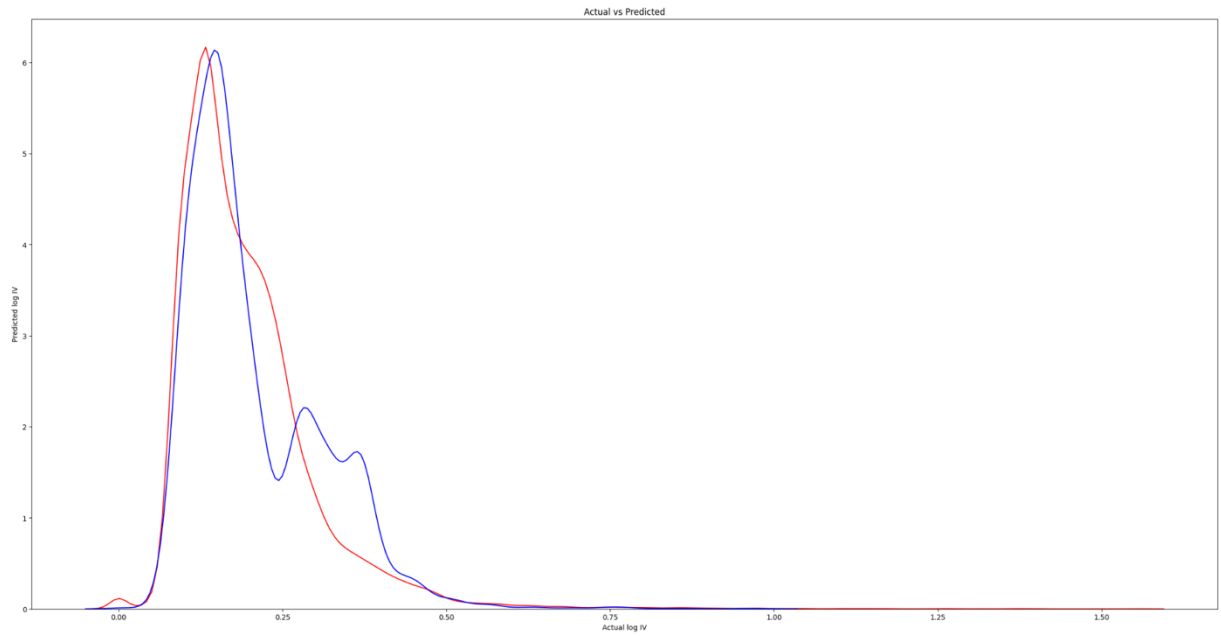


figure f. Scatterplot of Actual vs Predicted values of log IV

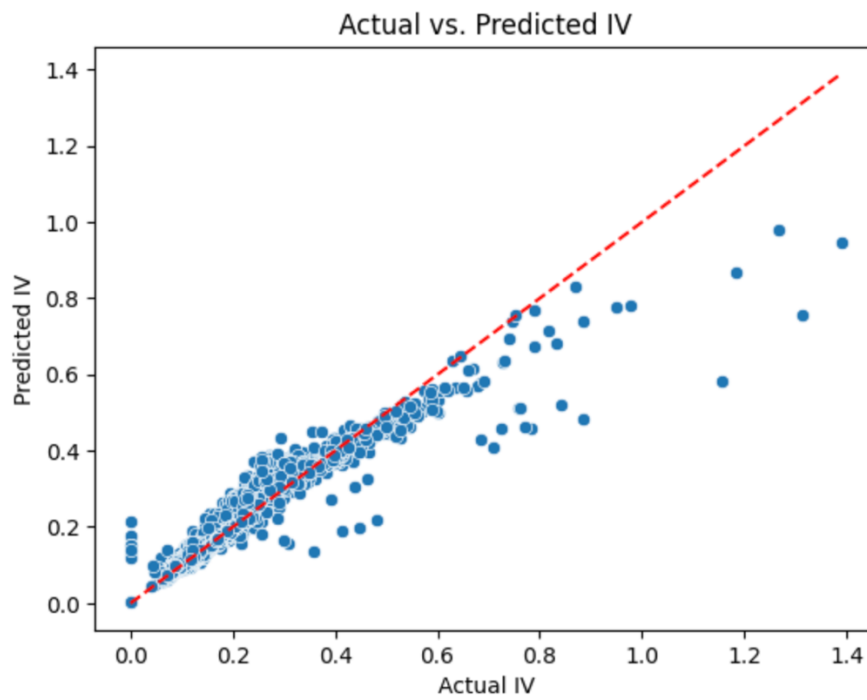


figure g. Scatterplot of Residuals vs. Predicted log IV

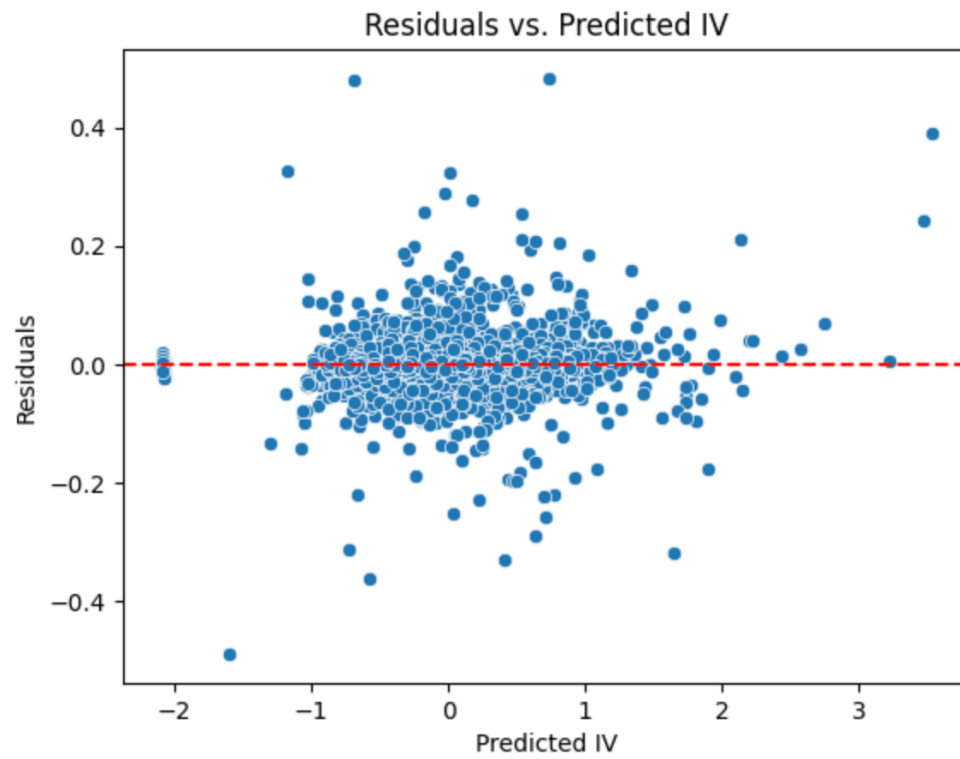
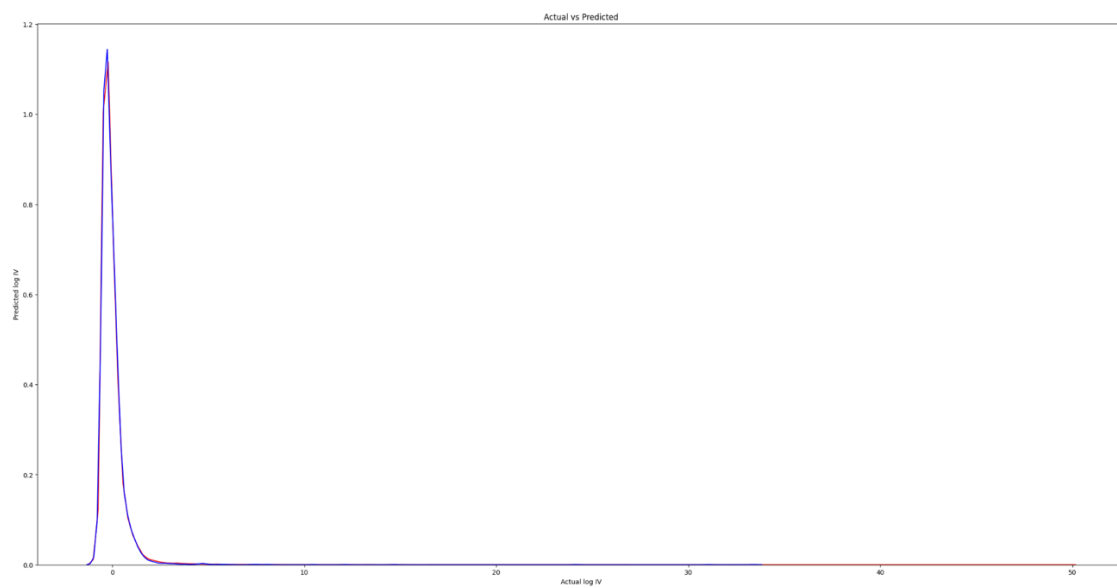


figure h. Distplot of Actual vs Precited values of IV



3. Exploratory Data Analysis (EDA)

The skewness in IV was exaggerating the MSE in the model and can be seen in the histogram above (figure a). To address the issue, the model uses log transformation on IV creating a new column `log_c_iv` as the target variable (figure d).

The correlation heatmap (figure c) shows that `log_c_iv` is most strongly associated with variables such as `c_vega`, `c_theta`, `dte`, and `log_moneyness`. These relationships align with financial theory, where time to expiry and sensitivity to volatility influence implied volatility levels.

4. Methodology and Implementation

1. Split features and target using `scikit-learn train_test_split`
2. Train model on basic Random Forest (`scikit-learn`) – The decision to use Random Forest came from further research and the Pearson correlation factors calculated that showed that no feature showed any linear correlation to the Implied Volatility. But a `mutual_info_regression` quickly revealed that there was non-linear correlation between features – implying that a tree-based approach would work best for the model.
3. Tune hyperparameters using `GridSearchCV` (`scikit-learn`)
4. Evaluate Model using MSE, MAE and R2 score

5. Results & Conclusion

The model worked appropriately after addressing the skewness in data as well as removing outliers. The final MAE, MSE and R2 scores were 0.0207917, 0.001434 and 0.867760 respectively indicating a strong fit and practical accuracy. These results indicate that the model captures a substantial portion of the variance in implied volatility, demonstrating strong predictive performance. The relatively low MAE and MSE suggest that, on average, the predicted values are close to the actual market-implied volatilities, making the model potentially

useful for tasks such as risk management, pricing, or volatility forecasting. However, an R^2 below 1 also reflects the inherent uncertainty and noise present in market data, underscoring that while the model performs well, there is still room for improvement through enhanced feature engineering, better modeling of market microstructure, or incorporating temporal dynamics.

```
[315]: # Evaluate the model
mae = mean_absolute_error(y_test, y_pred_basic)
mse = mean_squared_error(y_test, y_pred_basic)
r_squared = r2_score(y_test, y_pred_basic)

print("MAE:", mae)
print("MSE:", mse)
print("R2:", r_squared)
```

```
MAE: 0.02079179682854572
MSE: 0.001434640619374428
R2: 0.8677609169581517
```

6. Challenges & Limitations

Hyperparameter tuning resulted in slightly worse evaluation metrics as shown below. This can point to the model overfitting or data leakage. Another issue with the underlying dataset is that the Implied Volatility data was heavily skewed, possibly due to Q1 2020 during Covid-19. This was addressed using a log-transformation however the unpredictable market behavior may not be properly accounted by this model trying to predict implied volatility using constant attributes like strike price, date to expiry and other derived features.

```
: # Evaluate the model
mae = mean_absolute_error(y_test, y_pred_tuned)
mse = mean_squared_error(y_test, y_pred_tuned)
r_squared = r2_score(y_test, y_pred_tuned)

print("MAE:", mae)
print("MSE:", mse)
print("R2:", r_squared)
```

```
MAE: 0.020506358983685292
MSE: 0.001441409083249533
R2: 0.867137028686512
```


7. Future Work

In the next steps for this project, Cross Validation while splitting the dataset into training and testing sets can reduce overfitting and provide better well-rounded results. In addition, testing the model on realized volatility in a different dataset can help determine if the model is practical for real world use cases and can provide a better insight into the market volatility. The model can also improve by a functionality to back-test on historical data of options prices using the black-scholes equation. While the Random Forest model provides strong baseline performance, more advanced methods like XGBoost or GARCH can offer further improvements. XGBoost is a gradient boosting algorithm which often outperforms Random Forests by sequentially correcting errors from previous trees and optimizing the bias and variance. It's particularly effective when subtle nonlinear interactions or feature importance weighting are critical. On the other hand, GARCH models specialize in modeling and forecasting time-varying volatility, making them well-suited for financial time series. Incorporating GARCH can help capture the temporal dynamics if the implied volatility is modeled over time. The future work for this project would include using these approaches to increase the accuracy of the predictions.